

CSE234 Week 3: STL Vector and Iterators

This week we will be discussing the STL Vector class. A vector is an automatically resizing array. It is the next sequential container provided by the STL that we will discuss. We will also discuss iterators, which are used to traverse containers.

Vector

The STL Vector is an automatically resizing dynamic array wrapper. Unlike the STL Array, which requires the size to be known at compile time, when a vector runs out of space to hold new elements, it automatically resizes itself. Most implementations of vector double its size each time it runs out of space. Items can be inserted into the beginning or middle of a vector while preserving order. To do so, all elements that come after the inserted element must be shifted. This shift happens automatically, as well.

Construction/Initialization

Vectors must have the type of data it is storing specified as a template argument. Vectors do not have the size as a template argument like the STL Array did. An initial size can be provided to the constructor, however. Vectors support copying and assignment, like all STL containers. Like a C-array, a vector can be instantiated with a sequence of values in curly braces.

```
// need to include vector
#include <vector>

int main() {
    // declaring an empty vector of integers
    std::vector<int> integers;
    // declaring vector with memory for 10 doubles allocated
    std::vector<double> doubles(10);
    // declaring vector of c-strings
```

```

std::vector<const char*> months = {
    "January", "February", "March", "April",
    "May", "June", "July", "August", "September",
    "October", "November", "December"
};
// creating copy of months vector
// C++17 added type deduction, so you don't need to use
// the template argument when it is obvious what the type should
std::vector monthsCopy(months);
// you can also use the equals sign to assign vectors
// but they must have the same type
std::vector<int> otherIntegers;
otherIntegers = integers;
// can't do this, types mismatch
// otherIntegers = doubles;
}

```

Adding Elements

The main way to add new elements to the end of a vector is via the `push_back` function. Using the subscript `[]` operator to add new elements is undefined behavior. It may work, it may not, depending on the compiler, and should be avoided.

```

#include <iostream>
#include <vector>

int main() {
    std::vector<double> scores;
    double score = 0;

    while (score >= 0) {
        std::cout << "Enter score: ";
        std::cin >> score;
        scores.push_back(score);
    }

    return 0;
}

```

Accessing Elements

Vectors offer random access of its elements via the subscript operator `[]`. It is still possible to pass out of range elements to the subscript operator, just like a normal array. A member function `at(size_t index)` is used to safely access elements. If the value of `index` is out

of range, an exception is thrown. Some compilers may implement the subscript operator to throw an exception for out-of-range accesses like the `at()` function does, but this is not guaranteed. The size of the vector is accessed via the `size()` member function. The `capacity()` member function returns the maximum number of elements that are currently allocated for the vector. The first element of the vector can be accessed via the `front()` member function, and the last element is accessed via the `back()` member function. If you need access to the underlying c-array, the `data()` member function can be used. The `pop_back()` function can be used to remove the last element of the vector.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<double> scores;
    double score = 0;

    while (score >= 0) {
        std::cout << "Enter score: ";
        std::cin >> score;
        scores.push_back(score);
    }

    std::cout << "There are " << scores.size() << " scores in the vect
    std::cout << "First score: " << scores.front() << "\n";
    std::cout << "Last score: " << scores.back() << "\n";
    std::cout << "There are " << scores.capacity() << " elements reser

    for (int i = 0; i < scores.size(); ++i) {
        // accessing scores with the subscript operator
        std::cout << "Score " << i + 1 << ": " << scores[i] << "\n";
    }

    int lastIndex = scores.size() - 1;
    scores.pop_back();
    // won't throw an exception, but data could be invalid
    double lastScore = scores[lastIndex];
    // this will throw an exception
    lastScore = scores.at(lastIndex);

    return 0;
}
```

Before talking about the rest of the vector member functions, we need

to discuss iterators.

Iterators

Before discussing the rest of the STL data types, and the rest of the vector member functions, we need to cover iterators. An iterator is a type of pointer that was developed to standardize access to containers. An iterator is basically a pointer to an element of a container, however, there are additional operations that can be performed on iterators to make traversing a container consistent, no matter which type of container is being accessed. Below is a quick example of a vector being created, and an iterator being used to access its members:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    // the begin() member function returns an iterator to the first
    // element of a collection
    std::vector<int>::iterator it = values.begin();

    // we often use the "auto" type for iterators rather than writing
    // the end() member function returns the "past-the-end" iterator
    // of a collection. this iterator points to the location immediate
    // after the end of a collection
    auto end = values.end();

    // the end iterator is not dereferenceable, it is just a sentinel
    // value that marks the end of the container
    while (it != end) {
        // accessing the data an iterator is pointing to requires it
        // to be dereferenced using *, just like a pointer
        std::cout << *it << std::endl;
        // ++ is used to move the iterator to point at the next element
        ++it;
    }
    return 0;
}
```

As well as supporting the increment operator, normal addition can be performed on iterators as well. For example, if we wanted to access the 4th element of the above vector `values`, we could add 3 to the `begin()` iterator: `auto fourthElement = values.begin() + 3;`. This is because the `begin()` returns an iterator to the 0-index item.

The fourth element has the index of 3, so 0 + 3 is the iterator to the fourth element.

Range-based for-Loops

One consequence of iterators is the ability to implement range-based for-loops. If you are familiar with for loops in Python or the `foreach` loop in C#, this concept should be familiar to you. The syntax for a range-based for-loop is below:

```
for (TypeName x : container) {  
  
}
```

Below shows a traditional for-loop and a range-based for loop that performs the same operation:

```
#include <iostream>  
#include <vector>  
  
int main() {  
    std::vector<int> values = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };  
  
    // tradition for loop  
    for (int i = 0; i < values.size(); ++i) {  
        std::cout << values[i] << std::endl;  
    }  
  
    // range-based for loop  
    for (int x: values) {  
        std::cout << x << std::endl;  
    }  
  
    return 0;  
}
```

Vectors (continued)

Iterators can be used to insert at, or delete elements from, the middle of the vector. These operations are expensive, however, since they require shifting the entire vector to adjust for the missing or new element. The `insert()` function takes an iterator that points to the location the element will be inserted, and the element to be inserted. The element is inserted before the iterator the `insert()` function is

passed. The function returns an iterator to the element that was just inserted. The `erase()` function is used to remove item(s) from a vector. It expects an iterator that points to the element to be removed. A start and end iterator can be used to erase a range of elements from the vector. It returns an iterator to the next element after the last element that was removed.

Note: `insert()` and `erase()` invalidate any existing iterators to a vector. Since the items must be shifted, there is no guarantee that an iterator is pointing to the correct location in memory.

```
#include <vector>

int main() {
    std::vector<int> values = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    auto it = values.begin() + 2; // points to "3"
    values.insert(it, 2);
    // vector is now {1, 2, 2, 3, 4, 5, 6, 7, 8, 9, 10}
    // "it" is now invalid, so we need to reassign it
    it = values.begin() + 2;
    values.erase(it); // deletes the 2 that was just inserted
    // vector is now {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, x}
    // the last element is invalid, but still has memory allocated
    values.erase(values.begin(), values.begin() + 5); // erase first 5
    // vector is now {6, 7, 8, 9, 10}
}
```

Other Types of Iterators

Depending on the type of container, other iterators may be available. All STL containers support forward and constant forward iterators. Ordered containers like array and vector support reverse and constant reverse iterators as well. `cbegin()` and `cend()` are used for constant iterators. `rbegin()` and `rend()` are used for reverse iterators. `crbegin()` and `crend()` are used for constant reverse iterators.

Common Applications of Vectors

`std::vector` is the standard container used in C++ for storing collections of data. If you aren't sure which of the other containers to use for an application, that means a vector is likely best. Vectors offer balanced performance for most applications.

Example

This example uses vectors to split a string and store the contents of a CSV file that contains information on orders. CSV stands for comma separated values, and is a very commonly used data transfer format. Tabular data, like a spreadsheet or SQL table is represented well by a CSV file.

```
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

// we expect the structure of the file to be
// John Doe,3,1000a,Filter

// struct to hold order information
struct Order {
    std::string customerName;
    int quantity;
    std::string itemId;
    std::string itemName;
};

// overloading << to print an order
std::ostream& operator<<(std::ostream& out, const Order& order) {
    out << "Customer: " << order.customerName << "\n";
    out << "Quantity: " << order.quantity << "\n";
    out << "Item ID: " << order.itemId << "\n";
    out << "Item Name: " << order.itemName << "\n";
    return out;
}

// function to split a string on any character
std::vector<std::string> split(const std::string& line, char delim) {
    std::vector<std::string> parts;
    size_t start = 0;
    // find index delimiter in string
    size_t end = line.find(delim, start);
    // while end position of substring is not the past the end
    // of the string
    while (end != std::string::npos) {
        // push substring into vector
        parts.push_back(line.substr(start, end - start));
        // set the start of the next substring to the
        // next index of the string
        start = end + 1;
        // search for the next occurrence of "delim"
```

```

        end = line.find(delim, start);
    }
    // push the last substring into the vector
    // if "delim" is never found, it will just
    // push the whole string into the vector
    parts.push_back(line.substr(start, end - start));
    return parts;
}

// main function
int main(int argc, char **argv) {
    // input filename needs to be provided via command line arg
    if (argc < 2) {
        std::cerr << "Must provide input filename\n";
        std::cerr << "Usage:\n";
        std::cerr << "\torder_report orders.csv\n";
        return 1;
    }

    // open and check the input file
    std::ifstream fin(argv[1]);
    if (fin.fail()) {
        std::cerr << "Could not open file: " << argv[1] << "\n";
        return 2;
    }

    // read the lines of the file into the split function
    // convert them to orders and add to the orders vector
    std::vector<Order> orders;
    for (std::string line; std::getline(fin, line);) {
        std::vector<std::string> orderParts = split(line, ',');
        // verify that the input is not malformed and we have the corr
        // number of parts
        if (orderParts.size() != 4) {
            continue;
        }
        orders.push_back({
            orderParts[0],
            std::stoi(orderParts[1]),
            orderParts[2],
            orderParts[3]
        });
    }

    // done with file stream -> close it
    fin.close();

    // print out the orders
    std::cout << "Orders:\n";
    // using range-based for loop to access elements
    for (const auto& order: orders) {

```



```
        std::cout << "-----\n";
        std::cout << order;
        std::cout << "-----\n";
    }
    return 0;
}
```

Conclusion

Vectors are automatically resizing arrays. They support fast insertion and removal at the end of the container via the `push_back()` and `pop_back()` member functions. The subscript operator can be used to access individual elements of the vector. The `at()` member can be used to throw an exception if an out-of-range index is accessed. The `insert()` and `erase()` member functions are used to insert and remove elements from the middle of the vector, but are slow since the elements of the vector need to be shifted to compensate. Iterators are abstractions that allow for traversing containers. An iterator is essentially a pointer to an element of the container, but with the ability to be incremented to access the next element.